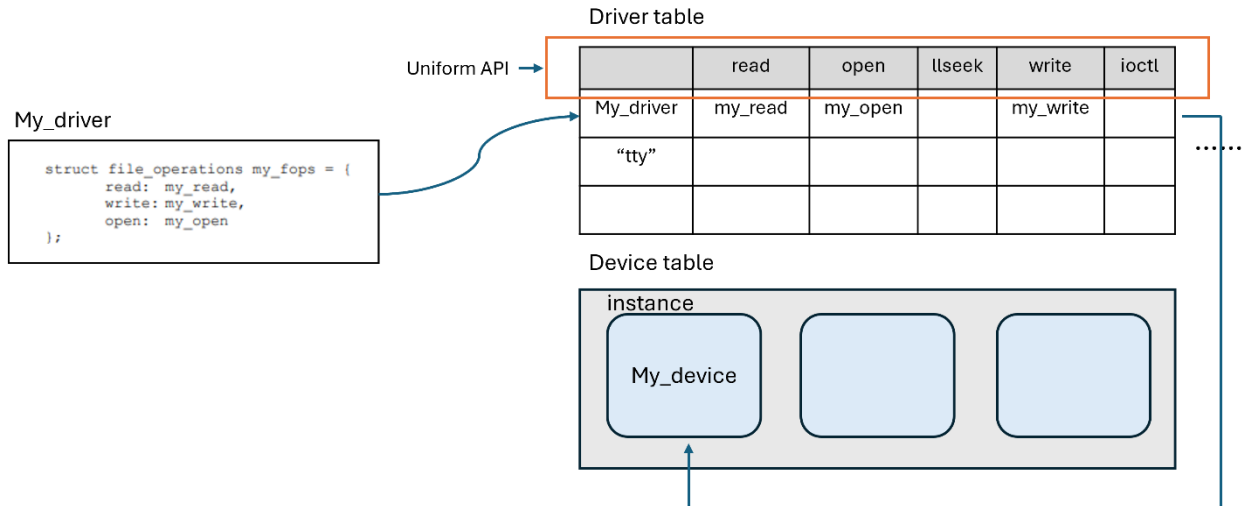


Lab4: Driver(Part 2)

313513032 王承翰

1. Driver table 與 Device table



2. Basic Driver

一個簡單又基本的 driver 如下

```
#include <linux/module.h>

static int __init hello_world_init (void)
{
    pr_info("Hello, World!\n");
    return 0;
}

static void __exit hello_world_exit (void)
{
    pr_info("Goodbye, World!\n");
}

/* 不可省略的 my 可省的 */
module_init(hello_world_init);
module_exit(hello_world_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Dimitri Kokkonis (kokkonisd@gmail.com)");
MODULE_DESCRIPTION("A simple hello world kernel module");
```

又可以分成以下三種狀況

- host 上編譯 host 上 insmod
- target(Rpi)上編譯 target(Rpi) insmod
- host 上編譯 target(Rpi)上 insmod

前兩者種況如 Makefile 與執行結果如下

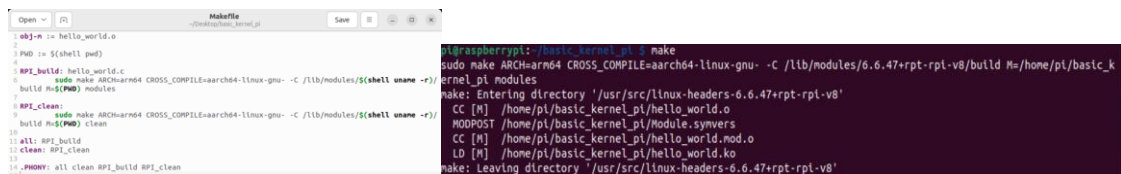
- host 上編譯 host 上 insmod

```
Makefile
~/Desktop/basic_kernel
Save
Makefile
hello_world.c
1 cc-gcc -lz
2 obj-m += hello_world.o
3
4 all: make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
5
6 clean: make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
7
8
9

root@EOS:/home/wang/Desktop/basic_kernel# make CC=gcc-12
make -C /lib/modules/6.8.0-40-generic/build M=/home/wang/Desktop/basic_kernel modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-40-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
You are using:      gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
CC [M] /home/wang/Desktop/basic_kernel/hello_world.o
MODPOST /home/wang/Desktop/basic_kernel/Module.symvers
CC [M] /home/wang/Desktop/basic_kernel/hello_world.mod.o
LD [M] /home/wang/Desktop/basic_kernel/hello_world.ko
BTF [M] /home/wang/Desktop/basic_kernel/hello_world.ko
Skipping BTF generation for /home/wang/Desktop/basic_kernel/hello_world.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-40-generic'
```

```
wang@EOS:~/Desktop/basic_kernel$ sudo insmod hello_world.ko
wang@EOS:~/Desktop/basic_kernel$ sudo dmesg|tail
[ 99.534792] rfkill: input handler disabled
[ 100.857709] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 100.857808] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 101.435145] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 101.435258] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 101.592073] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 106.711783] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 106.711999] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 116.556431] workqueue: wait_rcu_exp_gp hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 199.994044] Hello, World!
wang@EOS:~/Desktop/basic_kernel$ sudo rmmod hello_world.ko
wang@EOS:~/Desktop/basic_kernel$ sudo dmesg|tail
[ 100.857709] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 100.857808] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 101.435145] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 101.435258] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 101.592073] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 106.711783] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 106.711999] [drm:vmw_msg_ioctl [vmwgfx]] *ERROR* Failed to open channel.
[ 116.556431] workqueue: wait_rcu_exp_gp hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
[ 199.994044] Hello, World!
[ 215.985012] Goodbye, World!
wang@EOS:~/Desktop/basic_kernel$
```

- target(Rpi)上編譯 target(Rpi) insmod



```
Makefile
# obj-m := hello_world.o
#
# PWD := $(shell pwd)
#
# RPI_build: hello_world.c
#     sudo make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /lib/modules/$(shell uname -r)/
#     build M=$(PWD) modules
#
# RPI_clean:
#     sudo make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /lib/modules/$(shell uname -r)/
#     build M=$(PWD) clean
#
# all: RPI_build
#
# clean: RPI_clean
#
# PHONY: all clean RPI_build RPI_clean
```

```
pi@raspberrypi:~/basic_kernel_pi$ make
sudo make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /lib/modules/6.6.47+rpt-rpi-v8/build M=/home/pi/basic_k
make: Entering directory '/usr/src/linux-headers-6.6.47+rpt-rpi-v8'
CC [M] /home/pi/basic_kernel_pi/hello_world.o
MODPOST /home/pi/basic_kernel_pi/Module.symvers
CC [M] /home/pi/basic_kernel_pi/hello_world.mod.o
LD [M] /home/pi/basic_kernel_pi/hello_world.ko
make: Leaving directory '/usr/src/linux-headers-6.6.47+rpt-rpi-v8'
```

```
pi@raspberrypi:~/basic_kernel_pi$ sudo insmod hello_world.ko
pi@raspberrypi:~/basic_kernel_pi$ dmesg | tail
[ 8.353157] Bluetooth: hci0: BCM43455 37.4MHz Raspberry Pi 3+-0190
[ 8.353177] Bluetooth: hci0: BCM4345C0 (003.001.025) build 0382
[ 8.427925] Bluetooth: MGMT ver 1.22
[ 8.436649] NET: Registered PF_ALG protocol family
[ 9.974237] Bluetooth: RFCOMM TTY layer initialized
[ 9.974267] Bluetooth: RFCOMM socket layer initialized
[ 9.974287] Bluetooth: RFCOMM ver 1.11
[ 54.612444] warning: 'iwconfig' uses wireless extensions which will stop working for Wi-Fi 7 hardware; use nl
80211
[ 4403.639320] hello_world: loading out-of-tree module taints kernel.
[ 4403.639661] Hello, World!
pi@raspberrypi:~/basic_kernel_pi$ sudo rmmod hello_world.ko
pi@raspberrypi:~/basic_kernel_pi$ sudo rmmod hello_world.ko
pi@raspberrypi:~/basic_kernel_pi$ dmesg | tail
[ 8.353177] Bluetooth: hci0: BCM4345C0 (003.001.025) build 0382
[ 8.427925] Bluetooth: MGMT ver 1.22
[ 8.436649] NET: Registered PF_ALG protocol family
[ 9.974237] Bluetooth: RFCOMM TTY layer initialized
[ 9.974267] Bluetooth: RFCOMM socket layer initialized
[ 9.974287] Bluetooth: RFCOMM ver 1.11
[ 54.612444] warning: 'iwconfig' uses wireless extensions which will stop working for Wi-Fi 7 hardware; use nl
80211
[ 4403.639320] hello_world: loading out-of-tree module taints kernel.
[ 4403.639661] Hello, World!
[ 4677.611068] Goodbye, World!
pi@raspberrypi:~/basic_kernel_pi$
```

我們著重在介紹第三種狀況，也就是 CROSS_COMPILE，需要確定 target 的 kernel 版本，並下載相對應的 cross compiler tool chain，Makefile 如下：

```

obj-m := hello_world.o

all: x86_build #or RPI_build, you can specify only one build at a time

clean: x86_clean RPI_clean

x86_build: hello_world.c
    sudo make -C /lib/modules/5.4.0-52-generic/build M=$(PWD) modules

x86_clean:
    sudo make -C /lib/modules/5.4.0-52-generic/build M=$(PWD) clean

RPI_build: hello_world.c
    sudo make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C $(PWD)/linux M=$(PWD) modules

RPI_clean:
    sudo make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C $(PWD)/linux M=$(PWD) clean

.PHONY: all clean x86_build x86_clean RPI_build RPI_clean

```



在 `make` 之前記得先下指令確定工作目錄，以我的例子即為 `PWD=/home/wang/linux`，`$(PWD)`即為 Present Working Directory，否則無法找到相對應的 header file 還有 cross compiler。

```

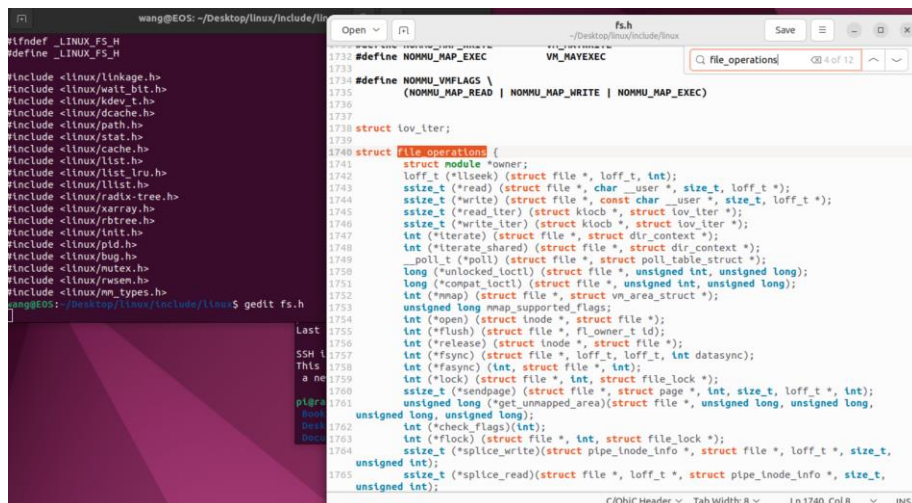
wang@EOS:~/linux$ PWD=/home/wang/linux
wang@EOS:~/linux$ make
sudo make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -C /home/wang/linux M=/home/wang/Desktop/basic_kernel_pi modules
make: Entering directory '/home/wang/linux'
CC [M] /home/wang/Desktop/basic_kernel_pi/hello_world.o
MODPOST /home/wang/Desktop/basic_kernel_pi/Module.symvers
CC [M] /home/wang/Desktop/basic_kernel_pi/hello_world.mod.o
LD [M] /home/wang/Desktop/basic_kernel_pi/hello_world.ko
make: Leaving directory '/home/wang/linux'

```

接著透過 `scp` 將編譯完後的 kernel object(.ko file)傳至 Rpi。後續 `sudo insmod hello_world.ko` 並利用 `dmesg` 即可看到 kernel information，若需移除指令為 `sudo rmmod hello_world.ko`

3. Uniform API

Uniform API 可以想成 kernel 提供的接口，driver 要甚麼就定義出來然後接上去。Uniform API 定義在 `/include/linux/fs.h` 中



```
struct file_operations my_fops = {
    read: my_read,
    write: my_write,
    open: my_open
};
```

透過以上定義需要的 API(寫在 driver 中)。

4. 實作練習

首先我先簡單地寫了一些 write、read、open 的 driver 的空殼，如下

```
1 #include <linux/init.h>
2 #include <linux/module.h>
3 #include <linux/kernel.h>
4 #include <linux/fs.h>
5 MODULE_LICENSE("GPL");
6 // File Operations
7
8 static ssize_t my_read(struct file *fp, char *buf, size_t count, loff_t *fpos) {
9     printk("call read\n");
10    return count;
11 }
12 static ssize_t my_write(struct file *fp, const char *buf, size_t count, loff_t *fpos){
13     printk("call write\n");
14     return count;
15 }
16 struct file_operations my_fops = {
17     read: my_read,
18     write: my_write
19 };
20 #define MAJOR_NUM 255
21 #define DEVICE_NAME "my_test_device"
22
23 static int my_init(void) {
24     printk("call init\n");
25     if(register_chrdev(MAJOR_NUM, DEVICE_NAME, &my_fops) < 0) {
26         printk("Can not get major %d\n", MAJOR_NUM);
27         return (-EBUSY);
28     }
29     printk("My device is started and the major is %d\n", MAJOR_NUM);
30     return 0;
31 }
32 static void my_exit(void) {
33     unregister_chrdev(MAJOR_NUM, DEVICE_NAME);
34     printk("call exit\n");
35 }
36
37 module_init(my_init);
38 module_exit(my_exit);
```

但是在 `insmod` 失敗了因為我發現我一開始 `MAJOR_NUM` 用 244 但被用過了，可以使用 `cat /proc/devices` 看哪些 number 已經使用。一旦 `insmod` 成功則下一步會需要建立 device node，`sudo mknod /dev/my_test_device c 255 0` 建立，c 為 character device，255 為 `MAJOR_NUM`，0 為 `MINOR_NUM`。透過 `ls -l /dev` 看是否成功安裝節點。

```
pi@raspberrypi:~$ sudo mknod /dev/my_test_device c 255 0
pi@raspberrypi:~$ ls -l /dev
```

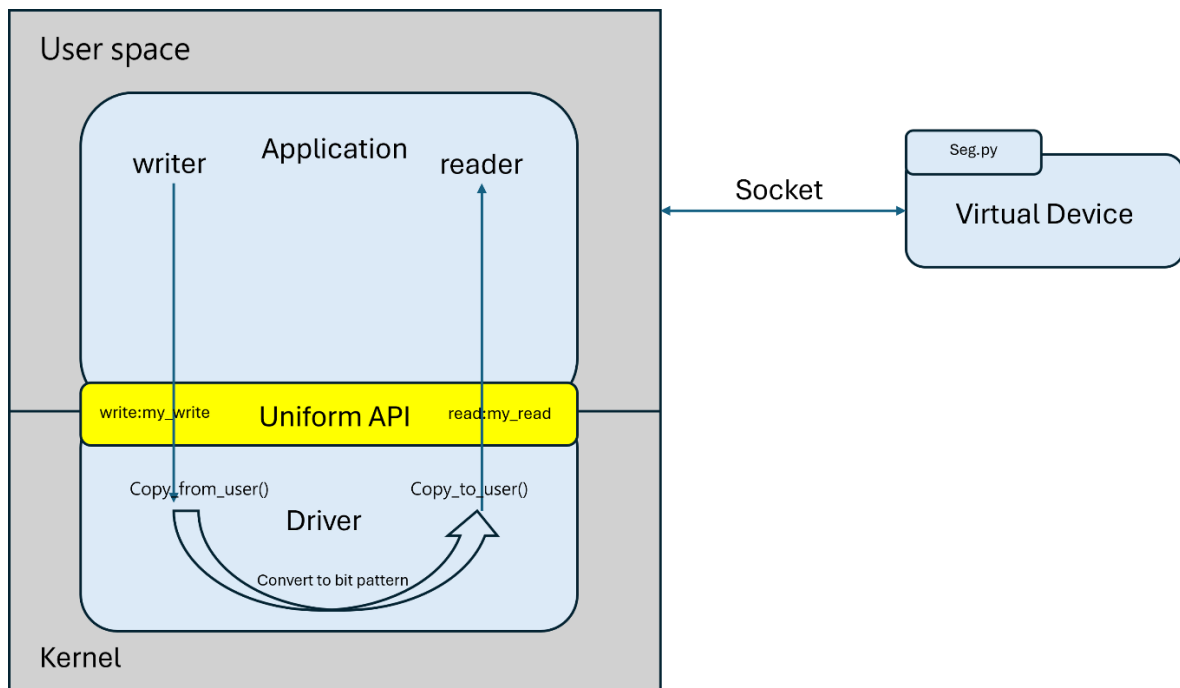
```

brw-rw---- 1 root disk 179, 1 Oct 5 20:29 mmcblk0p1
brw-rw---- 1 root disk 179, 2 Oct 5 20:29 mmcblk0p2
drwxrwxrwt 2 root root 40 Jan 1 1970 queue
crw-r--r-- 1 root root 255, 0 Oct 6 01:06 my_test_device
drwxr-xr-x 2 root root 60 Oct 5 20:29 net
crw-rw-rw- 1 root root 1, 3 Oct 5 20:29 null

pi@raspberrypi:~$ dmesg|tail
[ 274.525394] call init
[ 274.525408] Can not get major 244
[ 274.781872] call init
[ 274.781887] Can not get major 244
[ 364.554707] call init
[ 364.554721] Can not get major 244
[ 423.962535] call init
[ 423.962549] Can not get major 244
[ 792.637035] call init
[ 792.637051] My device is started and the major is 255

```

5. 本次 lab 架構圖



寫完 driver，insmod 和 mknod 後記得修改權限 `sudo chmod 666 /dev/my_device`，可以簡易的使用 `echo` 來測試是否成功利用 `copy_from_user()` 寫進 kernel。

```

pi@raspberrypi:~$ echo "123"> /dev/my_device
pi@raspberrypi:~$ dmesg|tail
[ 7.101334] Bluetooth: hci0: command 0x0c14 tx timeout
[ 7.101364] Bluetooth: hci0: BCM: Reading local name failed (-110)
[ 12.594261] IPv6: ADDRCONF(NETDEV_CHANGE): wlan0: link becomes ready
[ 245.626343] my_driver: loading out-of-tree module taints kernel.
[ 245.626625] module init
[ 245.626631] My device is started and the major is 255
[ 617.195569] my device open
[ 617.195608] my_write
[ 617.195611] kernel receive data: 123

```

完成 writer 後查看是否成功寫進

```

pi@raspberrypi:~/lab4 $ ./writer Devon
Wrote 5 bytes to the device: Devon
pi@raspberrypi:~/lab4 $ dmesg|tail -n 15
[11145.595198] my_write
[11145.595201] kernel receive data: D
[11146.595296] my_write
[11146.595307] kernel receive data: e
[11147.595391] my_write
[11147.595401] kernel receive data: v
[11148.595484] my_write
[11148.595494] kernel receive data: o
[11149.595575] my_write
[11149.595583] kernel receive data: n

```

在這邊我利用 `sleep(1)` 每秒寫一個字母進入 kernel 的 buffer 中，以讓 reader 每秒讀取一個字母。最後成功結果如下：

The image shows two terminal windows. The left window, titled 'pi@raspberrypi: ~/lab4', shows the execution of a C program that writes characters to a device. The output shows the characters 'D', 'e', 'v', 'o', 'n' being written. The right window, titled 'wang@EOS: ~/Desktop/lab4', shows the execution of a Python program that reads data from the device and displays it in a Tk window. The Tk window shows the characters 'D', 'e', 'v', 'o', 'n' being read.

6. Reference

<https://huenlil.pixnet.net/blog/post/23507650>
<https://blog.logan.tw/2013/01/linux-driver.html>
<https://www.apriorit.com/dev-blog/195-simple-driver-for-linux-os>
<https://fdgkhdkggh.medium.com/linux-lkmpg-%E7%B2%97%E6%B7%BA%E7%AD%86%E8%A8%98-1-256867ad6337>
<https://stackoverflow.com/questions/61908192/write-long-to-linux-char-device-driver>
https://jasonblog.github.io/note/linux_device_driver_programming/05.html

7. 問題與反思

- 可以利用 `echo` 進行簡單的 write 測試，有相對應的功能做 read 測試嗎
- 理論上要有 mutex 擋 reader 和 writer 同時做存取
- Socket 連線時有遇到可以 ping 的到卻連不到，結論是防火牆未關，需要下指令 `sudo ufw allow 8888/tcp`